

47. Un personnage de seize

Huit pixels de côté, c'est petit pour un héros.

1. Ce qu'il faut comprendre

Une `Tuile` fait huit pixels de côté. C'est la taille d'une case du décor, et c'est bien pour un mur ou une pièce — mais un personnage y tient mal. Un `Perso` en fait **seize sur seize** : seize rangées de seize signes, écrites comme celles d'une tuile.

Le matériel, lui, ne connaît que des carrés de huit. Un `Perso` occupe donc QUATRE numéros de tuile, dans cet ordre : haut-gauche, haut-droite, bas-gauche, bas-droite. Si `HEROS` vaut 44, il tient les numéros 44, 45, 46 et 47 — et le dessin suivant du programme commence à 48.

`sprite16(n, x, y, HEROS)` pose les **quatre lutins en un seul appel**, en carré, au pixel près. Le prix est dit franchement : il coûte quatre lutins des quarante, là où un `sprite()` n'en coûte qu'un. Une troupe de dix personnages de seize, c'est déjà tout le matériel. `cache16(n)` les ôte tous les quatre.

Sur le fond, il n'y a pas de `poser16()` — une case du décor fait huit pixels, et le personnage en couvre quatre. On pose donc ses quarts un par un, et l'ordre est celui des numéros : `poser(10, 8, HEROS); poser(11, 8, HEROS + 1); poser(10, 9, HEROS + 2); poser(11, 9, HEROS + 3);`. C'est ce que fait le programme ci-dessous, en plus du lutin.

Dans l'atelier, « + perso 16 × 16 » en ajoute un, et la grille passe à seize sur seize. Le reste ne change pas : on peint, et le programme se réécrit.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Perso>` → [35.1. Perso — dessiner un personnage de 16 × 16](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite16>` → [35.2. La fonction sprite16\(\) — un personnage de 16 × 16](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **746 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	61 o	← la console, à l'allumage — la mise en route de la console
main()	186 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran

```

InstallerTransfert    15 o ← démarage - installe la copie des lutins en mémoire rapide
CopierTuiles          85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte          14 o ← démarrage - vide les deux cartes de l'écran
LireManette           37 o ← bouton() - lit les huit boutons
Tuiles                240 o ← CopierTuiles - 11 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert      10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE"       13 o ← texte() - un texte gravé

```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   × 4   100 o sur place, dans main()
poser()     × 4    24 o sur place, dans main()
texte()     × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()    × 2    10 o sur place, dans main() + LireManette (37 o)
image()     × 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1     0 o sur place, dans main()

```

```

InstallerTransfert    15 o ← démarage - installe la copie des lutins en mémoire rapide
CopierTuiles          85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte          14 o ← démarrage - vide les deux cartes de l'écran
LireManette           37 o ← bouton() - lit les huit boutons
Tuiles                240 o ← CopierTuiles - 11 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert      10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE"       13 o ← texte() - un texte gravé

```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   × 4   100 o sur place, dans main()
poser()     × 4    24 o sur place, dans main()
texte()     × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()    × 2    10 o sur place, dans main() + LireManette (37 o)
image()     × 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1     0 o sur place, dans main()

```

```

InstallerTransfert    15 o ← démarage - installe la copie des lutins en mémoire rapide
CopierTuiles          85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte          14 o ← démarrage - vide les deux cartes de l'écran
LireManette           37 o ← bouton() - lit les huit boutons
Tuiles                240 o ← CopierTuiles - 11 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert      10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE"       13 o ← texte() - un texte gravé

```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   × 4   100 o sur place, dans main()
poser()    × 4    24 o sur place, dans main()
texte()    × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()   × 2    10 o sur place, dans main() + LireManette (37 o)
image()    × 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1     0 o sur place, dans main()

```

```

InstallerTransfert    15 o ← démarage - installe la copie des lutins en mémoire rapide
CopierTuiles          85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte          14 o ← démarrage - vide les deux cartes de l'écran
LireManette           37 o ← bouton() - lit les huit boutons
Tuiles                240 o ← CopierTuiles - 11 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert      10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE"       13 o ← texte() - un texte gravé

```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   × 4   100 o sur place, dans main()
poser()    × 4    24 o sur place, dans main()
texte()    × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()   × 2    10 o sur place, dans main() + LireManette (37 o)
image()    × 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1     0 o sur place, dans main()

```

```

InstallerTransfert    15 o ← démarage - installe la copie des lutins en mémoire rapide
CopierTuiles          85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte          14 o ← démarrage - vide les deux cartes de l'écran
LireManette           37 o ← bouton() - lit les huit boutons
Tuiles                240 o ← CopierTuiles - 11 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert      10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE"       13 o ← texte() - un texte gravé

```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

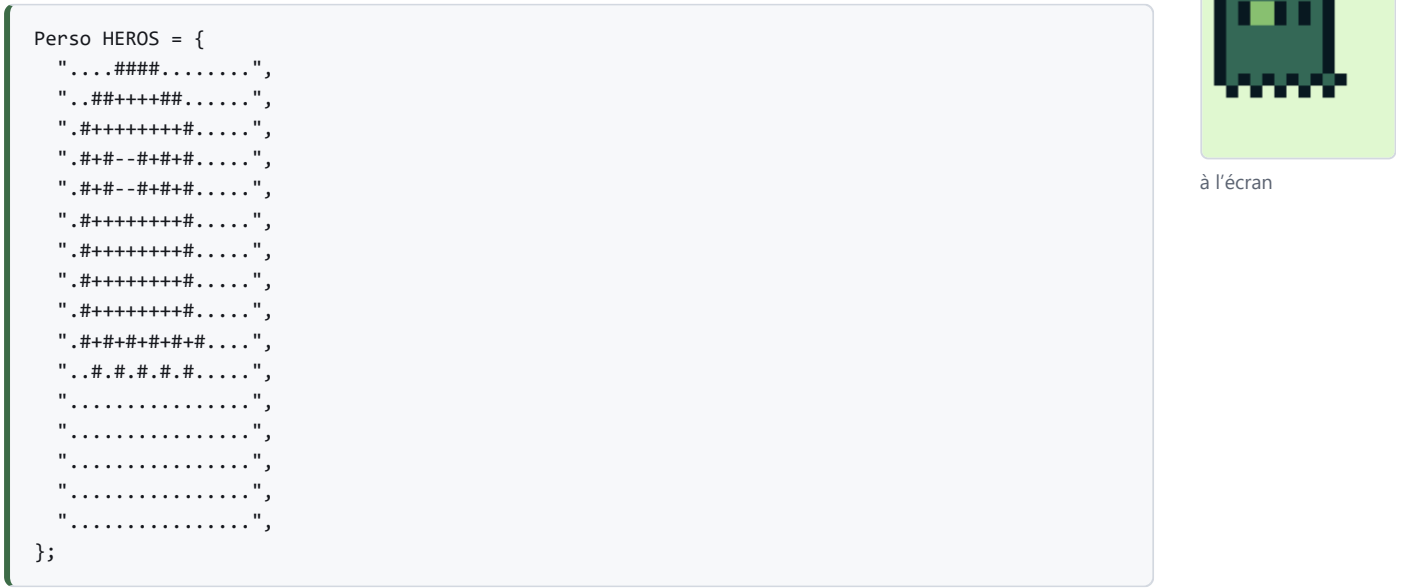
sprite()   × 4   100 o sur place, dans main()
poser()     × 4    24 o sur place, dans main()
texte()     × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()    × 2    10 o sur place, dans main() + LireManette (37 o)
image()     × 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1     0 o sur place, dans main()

```

```
#include <Perso>      // un dessin de 16 × 16 pixels, pour un lutin
#include <texte>       // écrit un texte à l'écran
#include <poser>       // pose une tuile sur une case du fond
#include <bouton>      // lit un bouton de la manette
#include <sprite16>    // place un lutin de 16 × 16 au pixel près
```

```
#include <Perso>      // un dessin de 16 × 16 pixels, pour un lutin
#include <texte>       // écrit un texte à l'écran
#include <poser>       // pose une tuile sur une case du fond
#include <bouton>      // lit un bouton de la manette
#include <sprite16>    // place un lutin de 16 × 16 au pixel près
```

```
Perso HEROS = {  
    "....####...",  
    "..#####...",  
    ".#+++++++#...",  
    ".#+##-#+##...",  
    ".#+##-#+##...",  
    ".#+++++++#...",  
    ".#+++++++#...",  
    ".#+++++++#...",  
    ".#+++++++#...",  
    ".#+###+###+###...",  
    "...#.#.#.#...",  
    ".....",  
    ".....",  
    ".....",  
    "....."  
};
```



```
Perso HEROS = {  
    "....####.....",  
    "..#####.....",  
    ".#++++++++#.....",  
    ".#+#- -#+#.....",  
    ".#+#- -#+#.....",  
    ".#++++++++#.....",  
    ".#++++++++#.....",  
    ".#++++++++#.....",  
    ".#++++++++#.....",  
    ".#.#.#.#.#.....",  
    ".....",  
    ".....",  
    ".....",  
    ".....",  
    "....."  
};
```



à l'écran

```
uint8_t x = 60;
```

```
uint8_t x = 60;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(1, 1, "GAUCHE DROITE");

    /* Le même personnage, POSÉ sur le fond : ses quatre quarts, un par case. */
    poser(10, 8, HEROS);
    poser(11, 8, HEROS + 1);
    poser(10, 9, HEROS + 2);
    poser(11, 9, HEROS + 3);

    while (true) {
        image();

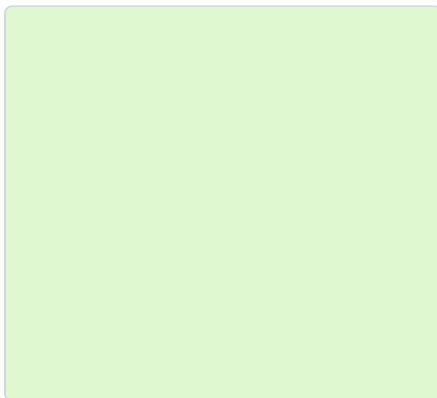
        if (bouton(DROITE) && x < 144) x++;
        if (bouton(GAUCHE) && x > 8) x--;

        /* En lutin : quatre lutins posés en carré, en un seul appel. */
        sprite16(0, x, 100, HEROS);
    }

    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros posé sur le décor, et le même en lutin plus bas — GAUCHE et DROITE le déplacent au pixel près.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ les quatre quarts sont posés sur le fond, dans l'ordre (tuiles 44, 45, 46, 47)
- ✓ `sprite16()` a rempli QUATRE lutins (tuiles 44, 45, 46, 47)
- ✓ ils sont posés en carré
- ✓ DROITE déplace le personnage entier (de 60 à 80)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

← l'atelier comp

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★



HEROS

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuiles n° 44 à 47 — sprite16(n, x, y, HERO

– zoom

11×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer le personnage



taille réelle — 16 × 16 pixels

étiquettes décor mur...

Animation

Un personnage s'anime dans La carte : Le joueur (la marche, et ses états — attente, course, attaque, blessure...) ou Les acteurs (« alterne avec »). Fais-lui d'abord ses images : « Dupliquer » HEROS, puis change la copie. Ici, ce sont les tuiles de 8 × 8 qui s'animent (l'eau, le feu, l'herbe).

Ensuite : 48. Déplacer un lutin — chapitre 7 : Le mouvement. · gameboy3 — écrire une cartouche Game Boy en C++